

源程序文档

接单吧 OPC撮合交易平台

软件版本：V1.0

编制日期：2026-07-23

源文件总数：394 个 代码总行数：125502 行

(依软著申请规范，本文档仅收录源代码前30页及后30页，共60页)

…… 中间省略约 385 个源文件（约 2307 页） ……

依国家版权局软件著作权申请规范：源程序超过60页时，提交前30页及后30页

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 if [[ -z "${GITHUB_TOKEN:-}" ]]; then
5     echo "■■■■GITHUB_TOKEN ■■■■"
6     exit 1
7 fi
8
9 REMOTE="https://ZhangColin:${GITHUB_TOKEN}@github.com/ZhangColin/opcorder.git"
10
11 git pull --no-rebase "$REMOTE" master
```

```
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  if [[ -z "${GITHUB_TOKEN:-}" ]]; then
5      echo "■■■■GITHUB_TOKEN ■■■■"
6      exit 1
7  fi
8
9  REMOTE="https://ZhangColin:${GITHUB_TOKEN}@github.com/ZhangColin/opcorder.git"
10
11  git push "$REMOTE" master
```

```
1 #!/bin/bash
2 set -e
3 pnpm install --frozen-lockfile
4 pnpm --filter db push --force
5 pnpm --filter @workspace/scripts exec tsx ./src/generate-schema-doc.ts
```

```

1  #!/usr/bin/env bash
2  # =====
3  # setup-github.sh
4  # ■■ GitHub ■■■HTTPS + Token ■■■■■/■■■
5  # ■ Replit Shell ■■■■■bash scripts/setup-github.sh
6  #
7  # ■■■GITHUB_TOKEN ■■■■■ Replit Secrets ■■
8  # =====
9
10 set -euo pipefail
11
12 GITHUB_USER="ZhangColin"
13 GITHUB_REPO="opcorder"
14 REMOTE_URL="https://${GITHUB_USER}:${GITHUB_TOKEN}@github.com/${GITHUB_USER}/${GITHUB_REPO}.git"
15
16 if [[ -z "${GITHUB_TOKEN:-}" ]]; then
17     echo "■■■GITHUB_TOKEN ■■■■■ Replit Secrets ■■■"
18     exit 1
19 fi
20
21 echo "=====
22 echo " ■■ GitHub ■■■HTTPS ■■■"
23 echo "=====
24
25 # ■■■■■ github ■■
26 if git remote get-url github &&/dev/null; then
27     echo "■■■ github ■■■..."
28     git remote set-url github "$REMOTE_URL"
29 else
30     echo "■■■ github ■■..."

```

```
31 git remote add github "$REMOTE_URL"
32 fi
33
34 echo "████████"
35 echo ""
36 echo "██████git push github master"
37 echo "██████git pull github master"
```

`scripts/sync-prod-to-test.sh`

```

1  #!/usr/bin/env bash
2  # =====
3  # sync-prod-to-test.sh
4  # ████████████████████████████████████████
5  #
6  # ████
7  # TEST_DATABASE_URL="postgresql://..." ./scripts/sync-prod-to-test.sh
8  #
9  # ████████
10 # ./scripts/sync-prod-to-test.sh "postgresql://user:pass@host/dbname"
11 #
12 # ████████████████████████████████████████
13 # =====
14
15 set -euo pipefail
16
17 PROD_DB="${DATABASE_URL:?████ DATABASE_URL ████████████████}"
18 TEST_DB="${1:-${TEST_DATABASE_URL:-}}"
19
20 if [[ -z "$TEST_DB" ]]; then
21     echo "████████████████████ TEST_DATABASE_URL ████████████████"
22     echo "████TEST_DATABASE_URL='postgresql://...' ./scripts/sync-prod-to-test.sh"
23     exit 1
24 fi
25
26 echo "======"
27 echo " █████ → ████████ ███"
28 echo "======"
29 echo "██████████${PROD_DB%?*}@***"
30 echo "██████████${TEST_DB%?*}@***"
31 echo ""
32 read -p "████████████████████████████████████████(y/N) " CONFIRM
33 if [[ "$CONFIRM" != "y" &&& "$CONFIRM" != "Y" ]]; then
34     echo "██████"
35     exit 0
36 fi
37
38 TIMESTAMP=$(date +%Y%m%dT%H%M%S)
39 DUMP_FILE="/tmp/prod_dump_${TIMESTAMP}.sql"
40
41 echo ""
42 echo "1/3 ████████████████████████████████████████..."
43 pg_dump \
44     --no-owner \
45     --no-acl \
46     --clean \
47     --if-exists \
48     "$PROD_DB" \
49     -f "$DUMP_FILE"
50
51 echo "2/3 ████████████████████████████████████████$(du -sh "$DUMP_FILE" | cut -f1)"
52
53 echo "3/3 ████████████████████████████████████████..."
54 psql "$TEST_DB" -f "$DUMP_FILE" -q
55
56 rm -f "$DUMP_FILE"

```

```
57
58 echo ""
59 echo "■■■■■$(date '+%Y-%m-%d %H:%M:%S')"
```

`scripts/src/backup-scheduler.ts`

```

1  /**
2   * backup-scheduler.ts
3   *
4   * Long-running process that triggers a PostgreSQL database backup every day
5   * at 03:00 local time and retains the last 30 days of backups.
6   *
7   * Scheduled via the "██████████" Replit workflow:
8   *   npm --filter @workspace/scripts exec tsx ./src/backup-scheduler.ts
9   */
10
11  import { execSync } from "child_process";
12  import { resolve, dirname } from "path";
13  import { fileURLToPath } from "url";
14
15  const __filename = fileURLToPath(import.meta.url);
16  const __dirname = dirname(__filename);
17  const WORKSPACE_ROOT = resolve(__dirname, "..", "..");
18  const BACKUP_SCRIPT = resolve(WORKSPACE_ROOT, "scripts/backup-db.sh");
19
20  // -----
21  // Helpers
22  // -----
23
24  function log(msg: string) {
25    console.log(`[backup-scheduler] ${new Date().toISOString()} ${msg}`);
26  }
27
28  /** Milliseconds until the next occurrence of HH:MM (today or tomorrow). */
29  function msUntil(targetHour: number, targetMin: number): number {
30    const now = new Date();
31    const next = new Date(now);
32    next.setHours(targetHour, targetMin, 0, 0);
33    if (next.getTime() <= now.getTime()) {
34      next.setDate(next.getDate() + 1);
35    }
36    return next.getTime() - now.getTime();
37  }
38
39  function humanDuration(ms: number): string {
40    const totalSec = Math.round(ms / 1000);
41    const h = Math.floor(totalSec / 3600);
42    const m = Math.floor((totalSec % 3600) / 60);
43    const s = totalSec % 60;
44    return `${h}h ${m}m ${s}s`;
45  }
46
47  // -----
48  // Backup runner
49  // -----
50
51  function runBackup() {
52    log("Running database backup...");
53    try {
54      execSync(`bash "${BACKUP_SCRIPT}"`, {
55        stdio: "inherit",
56        cwd: WORKSPACE_ROOT,
57        env: { ...process.env },
58      });
59      log("Backup completed successfully.");
60    } catch (err) {
61      log(`Backup failed: ${err}`);
62    }
63  }
64
65  // Run backup every day at 03:00
66  const nextRun = msUntil(3, 0);
67  log(`Next backup in: ${humanDuration(nextRun)}`);
68  runBackup();
69
70  // Exit the process
71  process.exit(0);

```

```

60   } catch (err) {
61     log(`ERROR: Backup failed - ${err as Error}.message`);
62     // Do not rethrow; keep the scheduler alive so the next run can still fire.
63   }
64 }
65
66 // -----
67 // Scheduler loop
68 // -----
69
70 const BACKUP_HOUR = 3;
71 const BACKUP_MIN = 0;
72
73 function scheduleNext() {
74   const ms = msUntil(BACKUP_HOUR, BACKUP_MIN);
75   const nextRun = new Date(Date.now() + ms);
76   log(
77     `Next backup scheduled for ${nextRun.toLocaleString()} ` +
78     `(in ${humanDuration(ms)})`
79   );
80
81   // setTimeout max is ~24.8 days; well within the 1-day interval, so this is safe.
82   setTimeout(() => {
83     runBackup();
84     scheduleNext(); // Reschedule for the following day.
85   }, ms);
86 }
87
88 // -----
89 // Entry point
90 // -----
91
92 log("Backup scheduler started.");
93 log(`Backup script: ${BACKUP_SCRIPT}`);
94 log(`Schedule: every day at ${String(BACKUP_HOUR).padStart(2, "0")}:${String(BACKUP_MIN).padStart(2, "0")} loc ...`);
95 log("Retention: 30 days (managed by backup-db.sh)");
96
97 scheduleNext();
98
99 // Keep the process alive (the setTimeout above already does, but this makes
100 // the intent explicit and survives any future refactor that removes it).
101 process.on("SIGTERM", () => {
102   log("Received SIGTERM, shutting down scheduler.");
103   process.exit(0);
104 });

```

scripts/src/generate-schema-doc.ts

```

1  /**
2   * generate-schema-doc.ts
3   *
4   * Reads every *.ts file under lib/db/src/schema/, parses pgTable definitions,
5   * and overwrites .local/docs/database-schema-zh.md with an up-to-date Chinese
6   * documentation table.
7   *
8   * Run:  npm run tsx scripts/src/generate-schema-doc.ts
9   */
10
11 import { readFileSync, writeFileSync, readdirSync } from "fs";
12 import { join, resolve, dirname } from "path";
13 import { fileURLToPath } from "url";
14
15 // -----
16 // Paths – resolved relative to this script file so they work regardless of CWD
17 // -----
18 const __filename = fileURLToPath(import.meta.url);

```

```

19 const __dirname = dirname(__filename);
20 // scripts/src/ → up two levels → workspace root
21 const WORKSPACE_ROOT = resolve(__dirname, "..", "..");
22 const SCHEMA_DIR = join(WORKSPACE_ROOT, "lib/db/src/schema");
23 const OUTPUT_FILE = join(WORKSPACE_ROOT, ".local/docs/database-schema-zh.md");
24
25 // -----
26 // Module groupings – defines the order and sections in the output document.
27 // Tables not listed here are appended to a catch-all section at the end.
28 // -----
29 const MODULES: Array<{ title: string; tables: string[] }> = [
30   {
31     title: "用户和交易",
32     tables: ["users", "opc_profiles", "publisher_profiles", "refresh_tokens", "settlement_accounts"],
33   },
34   {
35     title: "需求",
36     tables: ["demands", "demand_payments", "bids", "orders", "deliverables"],
37   },
38   {
39     title: "投资组合",
40     tables: ["portfolios"],
41   },
42   {
43     title: "报价",
44     tables: ["quote_dimensions", "quote_tiers", "quote_card_configs"],
45   },
46   {
47     title: "通知",
48     tables: ["notifications"],
49   },
50   {
51     title: "AI",
52     tables: ["llm_providers", "agent_configs", "agent_conversations"],
53   },
54   {
55     title: "课程",
56     tables: ["courses", "enrollments", "learning_resources"],
57   },
58   {
59     title: "活动",
60     tables: ["activities", "activity_fields", "registrations", "registration_tags"],
61   },
62   {
63     title: "帖子",
64     tables: ["posts", "post_likes", "post_comments", "announcements"],
65   },
66   {
67     title: "系统",
68     tables: ["admin_roles", "admin_role_assignments", "sensitive_words", "site_settings", "system_logs"],
69   },
70 ];
71
72 // -----
73 // Table metadata: Chinese name + one-line description
74 // -----
75 const TABLE_META: Record<string, { zh: string; desc: string }> = {
76   users: { zh: "用户", desc: "OPC平台用户" },
77   opc_profiles: { zh: "OPC 用户", desc: "OPC 用户详细配置" },
78   publisher_profiles: { zh: "发布者", desc: "发布者详细配置" },
79   refresh_tokens: { zh: "刷新令牌", desc: "JWT Refresh Token" },
80   settlement_accounts: { zh: "结算账户", desc: "OPC 结算账户/详细配置" },
81   demands: { zh: "需求", desc: "需求详细配置" },
82   demand_payments: { zh: "需求支付", desc: "需求支付详细配置" },
83   bids: { zh: "需求 / 报价", desc: "OPC 需求/报价" },

```



```

84 orders: { zh: "订单", desc: "订单" },
85 deliverables: { zh: "交付物", desc: "交付物" },
86 portfolios: { zh: "投资组合", desc: "投资组合" },
87 quote_dimensions: { zh: "报价维度", desc: "报价维度" },
88 quote_tiers: { zh: "报价 tiers", desc: "报价 tiers" },
89 quote_card_configs: {
90   zh: "报价卡配置",
91   desc: "报价卡配置 - 包含 `quote_dimensions` 和 `quote_tiers` 信息",
92 },
93 notifications: { zh: "通知", desc: "通知" },
94 llm_providers: { zh: "LLM 提供者", desc: "LLM 提供者" },
95 agent_configs: { zh: "AI 配置", desc: "AI 配置 Agent" },
96 agent_conversations: { zh: "AI 对话", desc: "AI 对话" },
97 courses: { zh: "课程", desc: "课程 OPC" },
98 enrollments: { zh: " enrollment", desc: " enrollment" },
99 learning_resources: { zh: "学习资源", desc: "学习资源" },
100 activities: { zh: "活动", desc: "活动" },
101 activity_fields: { zh: "活动字段", desc: "活动字段" },
102 registrations: { zh: "注册", desc: "注册" },
103 registration_tags: { zh: "注册标签", desc: "注册标签" },
104 posts: { zh: "帖子", desc: "帖子 OPC" },
105 post_likes: { zh: "帖子点赞", desc: "帖子点赞" },
106 post_comments: { zh: "帖子评论", desc: "帖子评论" },
107 announcements: { zh: "公告", desc: "公告" },
108 admin_roles: { zh: "管理员角色", desc: "管理员角色" },
109 admin_role_assignments: { zh: "管理员角色分配", desc: "管理员角色分配" },
110 sensitive_words: { zh: "敏感词", desc: "敏感词" },
111 site_settings: { zh: "网站设置", desc: "网站设置" },
112 system_logs: { zh: "系统日志", desc: "系统日志" },
113 };
114
115 // -----
116 // Column metadata: "table.column" → Chinese field description
117 // For columns not listed here, the column name itself is used as a fallback.
118 // -----
119 const FIELD_META: Record<string, string> = {
120   // ---- common ----
121   "*.id": "ID",
122   "*.created_at": "创建时间",
123   "*.updated_at": "更新时间",
124   "*.user_id": "用户 ID",
125   "*.status": "状态",
126   "*.is_active": "是否活跃",
127   "*.sort_order": "排序",
128   "*.description": "描述",
129   "*.title": "标题",
130   "*.file_url": "文件 URL",
131   "*.file_type": "文件类型",
132   "*.file_size": "文件大小",
133   "*.refund_reason": "退款原因",
134   "*.refund_order_no": "退款订单号",
135   "*.refunded_at": "退款时间",
136   "*.refund_requested_at": "退款请求时间",
137   "*.refund_reject_reason": "退款拒绝原因",
138
139   // ---- users ----
140   "users.nickname": "昵称",
141   "users.email": "邮箱",
142   "users.password_hash": "密码哈希",
143   "users.phone": "手机号",
144   "users.avatar": "头像 URL",
145   "users.title": "职位/头衔",
146   "users.role": "`opc` / `publisher` / `admin`",
147   "users.status": "`active` / `suspended` / `banned`",
148   "users.is super admin": "是否是超级管理员",

```

```
149
150 // ---- opc_profiles ----
151 "opc_profiles.id": "■■ ID",
152 "opc_profiles.user_id": "■■■■",
153 "opc_profiles.level": "OPC ■■■\`newbie` / `C` / `B` / `A`",
154 "opc_profiles.bio": "■■■■",
155 "opc_profiles.skill_tags": "■■■■■■",
156 "opc_profiles.industry_tags": "■■■■■■",
157 "opc_profiles.credit_score": "■■■■■■ 4.0",
158 "opc_profiles.total_orders": "■■■■■■",
159 "opc_profiles.completion_rate": "■■■■",
160 "opc_profiles.avg_rating": "■■■■",
161 "opc_profiles.total_earnings": "■■■■■■",
162 "opc_profiles.activity_score": "■■■■■■",
163 "opc_profiles.title": "■■/■■",
164 "opc_profiles.location": "■■■■",
165 "opc_profiles.website": "■■■■",
166 "opc_profiles.years_exp": "■■■■",
167 "opc_profiles.wechat": "■■■",
168 "opc_profiles.avatar": "■■ URL■■■■ users ■■",
169
170 // ---- publisher_profiles ----
171 "publisher_profiles.user_id": "■■■■■■■■",
172 "publisher_profiles.company_desc": "■■■■",
173 "publisher_profiles.location": "■■■■",
174 "publisher_profiles.industry": "■■■■",
175 "publisher_profiles.team_size": "■■■■",
176 "publisher_profiles.founded_year": "■■■■",
177 "publisher_profiles.website": "■■■■",
178 "publisher_profiles.contact_email": "■■■■",
179 "publisher_profiles.credit_code": "■■■■■■■■",
180 "publisher_profiles.company_logo": "■■ Logo URL",
181
182 // ---- refresh_tokens ----
183 "refresh_tokens.user_id": "■■■■■■■■■■■■■■■■",
184 "refresh_tokens.token_hash": "Token ■■■■■■■■■",
185 "refresh_tokens.expires_at": "■■■■",
186
187 // ---- settlement_accounts ----
188 "settlement_accounts.user_id": "■■■■",
189 "settlement_accounts.company_name": "■■■■",
190 "settlement_accounts.credit_code": "■■■■■■■■",
191 "settlement_accounts.bank_name": "■■■■■■",
192 "settlement_accounts.bank_branch": "■■■■",
193 "settlement_accounts.bank_account": "■■■■",
194 "settlement_accounts.account_name": "■■■",
195 "settlement_accounts.contact_name": "■■■■",
196 "settlement_accounts.contact_phone": "■■■■",
197 "settlement_accounts.business_license_url": "■■■■■■ URL",
198 "settlement_accounts.legal_rep_id_front_url": "■■■■■■ URL",
199 "settlement_accounts.legal_rep_id_back_url": "■■■■■■ URL",
200 "settlement_accounts.reject_reason": "■■■■■■",
201 "settlement_accounts.status": "■■■■\`pending` / `verified` / `rejected`",
202
203 // ---- demands ----
204 "demands.demand_no": "■■■■■■■■■■■■■■■■",
205 "demands.title": "■■■■",
206 "demands.type": "■■■\`education` / `software` / `marketing` / `content` / `other`",
207 "demands.description": "■■■■■■",
208 "demands.skill_tags": "■■■■■■",
209 "demands.opc_level": "■■■ OPC ■■■\`any` ■■■■■",
210 "demands.budget": "■■ ■■■■■■■",
211 "demands.budget_min": "■■■■■■■■",
212 "demands.budget_max": "■■■■■■■■",
213 "demands.deadline": "■■■■■■",
```

第 11 页

```

279 "deliverables.milestone_id": "██████ ID██████████████████",
280 "deliverables.title": "██████",
281 "deliverables.description": "██████",
282 "deliverables.file_url": "██████ URL",
283 "deliverables.file_name": "██████",
284 "deliverables.status": "████`submitted`██████/`approved`██████/`rejected`██████",
285 "deliverables.feedback": "██████/██████",
286 "deliverables.submitted_at": "██████",
287
288 // ---- portfolios ----
289 "portfolios.user_id": "██ OPC",
290 "portfolios.title": "██████",
291 "portfolios.type": "██████ education / software ███",
292 "portfolios.cover_image": "██████ URL",
293 "portfolios.description": "██████",
294 "portfolios.project_url": "██████",
295 "portfolios.order_id": "██████ ID██████████████████",
296 "portfolios.rating": "██████",
297 "portfolios.client_feedback": "██████",
298 "portfolios.apply_level": "██████████████A / B / C█",
299 "portfolios.level_apply_status": "████████████pending / approved / rejected█",
300 "portfolios.level_apply_note": "██████████████████",
301 "portfolios.reviewed_at": "██████",
302
303 // ---- quote_dimensions ----
304 "quote_dimensions.category": "██████`software` / `education` / `marketing` / `content` / `other`",
305 "quote_dimensions.layer": "██████`base`██████████/`adjust`██████████",
306 "quote_dimensions.code": "████████ D1█C1██████████",
307 "quote_dimensions.label": "██████████████",
308 "quote_dimensions.description": "██████",
309
310 // ---- quote_tiers ----
311 "quote_tiers.dimension_id": "██████",
312 "quote_tiers.tier": "████████ S / M / L / XL█",
313 "quote_tiers.tier_label": "██████████████",
314 "quote_tiers.base_price": "██████████████ base ███",
315 "quote_tiers.coefficient": "██████████ adjust ███ 1.2█",
316 "quote_tiers.description": "██████",
317
318 // ---- quote_card_configs ----
319 "quote_card_configs.dimension_code": "████████D1-D5 / C1-C4█",
320 "quote_card_configs.dimension_label": "██████",
321 "quote_card_configs.tier": "██████S / M / L / XL█",
322 "quote_card_configs.tier_label": "██████",
323 "quote_card_configs.base_price": "██████████",
324 "quote_card_configs.coefficient": "██████",
325 "quote_card_configs.description": "███",
326
327 // ---- notifications ----
328 "notifications.user_id": "██████████",
329 "notifications.type": "██████████████",
330 "notifications.title": "██████",
331 "notifications.content": "██████",
332 "notifications.is_read": "██████",
333 "notifications.responded_action": "██████████████/██████████",
334 "notifications.related_id": "██████ ID██████ ID██████ ID█",
335 "notifications.related_type": "████████████`order` / `demand`█",
336
337 // ---- llm_providers ----
338 "llm_providers.name": "██████████████`deepseek`█",
339 "llm_providers.display_name": "██████",
340 "llm_providers.base_url": "API █████",
341 "llm_providers.api_key": "API ███",
342 "llm_providers.default_model": "██████████████",
343 "llm_providers.remark": "██████",

```

```

344 // ---- agent_configs ----
345 "agent_configs.name": "████",
346 "agent_configs.scene_key": "██████████ `demand_analysis`█",
347 "agent_configs.system_prompt": "██████",
348 "agent_configs.is_enabled": "████",
349 "agent_configs.model": "██████████ deepseek-chat█",
350
351 // ---- agent_conversations ----
352 "agent_conversations.demand_id": "███████████",
353 "agent_conversations.session_key": "███████████████████████████████",
354 "agent_conversations.user_id": "████",
355 "agent_conversations.messages": "██████████████ `role`█ `content`███████",
356
357 // ---- courses ----
358 "courses.title": "████",
359 "courses.category": "███ `tech`████ / `strategy`████ / `compliance`████ / `operations`████",
360 "courses.required_level": "██████ `C` / `B` / `A`",
361 "courses.duration_minutes": "███████████",
362 "courses.description": "████",
363 "courses.badge": "███████████████",
364 "courses.rating": "████",
365 "courses.learners_count": "████",
366 "courses.is_required": "██████",
367 "courses.status": "███ `draft` / `published` / `closed`",
368 "courses.price": "██████0 █████",
369 "courses.syllabus_url": "████ URL",
370 "courses.instructor": "████",
371 "courses.max_enrollments": "███████████████",
372
373 // ---- enrollments ----
374 "enrollments.course_id": "████",
375 "enrollments.user_id": "████",
376 "enrollments.progress_pct": "██████████████0-100█",
377 "enrollments.completed_at": "████",
378 "enrollments.payment_status": "██████ `free` / `pending` / `paid` / `refund_pending` / `refunded`",
379 "enrollments.payment_order_no": "██████",
380 "enrollments.cert_issued": "██████████",
381 "enrollments.cert_issued_at": "███████",
382
383 // ---- learning_resources ----
384 "learning_resources.title": "████",
385 "learning_resources.file_url": "██ URL",
386 "learning_resources.file_type": "██████████ pdf / video█",
387 "learning_resources.file_size": "███████████",
388 "learning_resources.description": "████",
389
390 // ---- activities ----
391 "activities.title": "████",
392 "activities.description": "████",
393 "activities.location": "████",
394 "activities.start_time": "████",
395 "activities.end_time": "████",
396 "activities.status": "███ `draft` / `published` / `closed`",
397
398 // ---- activity_fields ----
399 "activity_fields.activity_id": "████",
400 "activity_fields.label": "███████████████████",
401 "activity_fields.field_type": "██████ `text` / `select` / `checkbox` █",
402 "activity_fields.is_required": "████",
403 "activity_fields.options": "██████select/checkbox █████",
404
405 // ---- registrations ----
406 "registrations.activity_id": "████",
407 "registrations.name": "██████",

```

```

409     "registrations.phone": "■■■■",
410     "registrations.email": "■■",
411     "registrations.organization": "■■■■",
412     "registrations.extra_data": "■■■■■■■■■■",
413     "registrations.admin_note": "■■■■■■",
414
415     // ---- registration_tags ----
416     "registration_tags.registration_id": "■■■■■■■■",
417     "registration_tags.tag": "■■■■",
418
419     // ---- posts ----
420     "posts.author_id": "■■",
421     "posts.title": "■■",
422     "posts.content": "■■■■■■■■",
423     "posts.tags": "■■■■",
424     "posts.likes_count": "■■■■",
425     "posts.comments_count": "■■■■",
426     "posts.views_count": "■■■■",
427     "posts.is_featured": "■■■■■■■■",
428
429     // ---- post_likes ----
430     "post_likes.post_id": "■■■■",
431     "post_likes.user_id": "■■■■",
432
433     // ---- post_comments ----
434     "post_comments.post_id": "■■■■",
435     "post_comments.author_id": "■■■■",
436     "post_comments.content": "■■■■",
437
438     // ---- announcements ----
439     "announcements.title": "■■",
440     "announcements.file_url": "■■ URL■■■■ PDF ■■",
441     "announcements.file_name": "■■■■■■■■",
442     "announcements.file_type": "■■ MIME ■■",
443     "announcements.is_pinned": "■■■■■■",
444
445     // ---- admin_roles ----
446     "admin_roles.name": "■■■■",
447     "admin_roles.description": "■■■■",
448     "admin_roles.permissions": "■■■■■■■■■■■■■■■■■■■■",
449
450     // ---- admin_role_assignments ----
451     "admin_role_assignments.user_id": "■■■■■■ ID■■■■■■■■",
452     "admin_role_assignments.role_id": "■■ ID■■■■■■■■",
453
454     // ---- sensitive_words ----
455     "sensitive_words.word": "■■■■■■■■",
456
457     // ---- site_settings ----
458     "site_settings.key": "■■■■■■■■",
459     "site_settings.value": "■■■■",
460
461     // ---- system_logs ----
462     "system_logs.level": "■■■■■■`info` / `warn` / `error`",
463     "system_logs.category": "■■■■■■`payment` / `user` / `demand`■■",
464     "system_logs.message": "■■■■",
465     "system_logs.metadata": "■■■■■■■■",
466     "system_logs.operator_id": "■■■■■■ ID■■■■■■■■■■■■■■■■■■■■",
467 };
468
469 // Extra per-table footnotes rendered below the column table
470 const TABLE_FOOTNOTES: Record<string, string> = {
471     demands:
472         "***■■■■■■■■■■■■■■■■■■■■**\n" +
473         "`draft`■■■■■■→ `pending_review`■■■■■■→ `pending_payment`■■■■■■→ `published`■■■■■■→ `matched`■■■■■■→ `in_pro"

```

```

474     "■■■■\`closed`■■■■/ \`refund_pending` / \`refunding` / \`refunded`■■■■",
475     notifications:
476     "***■■■■** \`bid_received`■■■■/ \`bid_accepted`■■■■/ \`bid_rejected`■■■■/ \`order_created`■■■■/ \`
477     admin_roles:
478     "***■■■■** \`dashboard`■■■■/ \`cockpit`■■■■/ \`users`■■■■/ \`demands`■■■■/ \`payments`■■■■/ \`orde
479 ];
480
481 // -----
482 // Parsed representation
483 // -----
484 interface ColumnDef {
485     colName: string;    // DB column name (snake_case)
486     typeSig: string;    // human-readable type string
487 }
488
489 interface TableDef {
490     tableName: string;  // DB table name
491     columns: ColumnDef[];
492     sourceFile: string;
493 }
494
495 // -----
496 // Parser helpers
497 // -----
498
499 /**
500  * Convert a JS type call chain into a human-readable type signature.
501  * e.g. "serial("id").primaryKey()" → "serial PK"
502  *      "integer("user_id").notNull().references(()) => usersTable.id" → "integer FK→users"
503  */
504 function buildTypeSig(typeExpr: string, allTables: Map<string, string>): string {
505     const isPK = typeExpr.includes(".primaryKey()");
506     const isUnique = typeExpr.includes(".unique()");
507
508     // FK detection
509     let fkTarget = "";
510     const refMatch = typeExpr.match(/\.references\s*(\s*(\s*\)\s*=>\s*(\w+)\.)/);
511     if (refMatch) {
512         const varName = refMatch[1];
513         fkTarget = allTables.get(varName) ?? varName.replace(/Table$/, "");
514     }
515
516     // Base type
517     let base = "unknown";
518     if (/^serial\b/.test(typeExpr)) base = "serial";
519     else if (/^integer\b/.test(typeExpr) || /^int\b/.test(typeExpr)) base = "integer";
520     else if (/^varchar\b/.test(typeExpr)) {
521         const lenMatch = typeExpr.match(/length:\s*(\d+)/);
522         base = lenMatch ? `varchar(${lenMatch[1]})` : "varchar";
523     } else if (/^text\b/.test(typeExpr)) base = "text";
524     else if (/^real\b/.test(typeExpr)) base = "real";
525     else if (/^boolean\b/.test(typeExpr)) base = "boolean";
526     else if (/^timestamp\b/.test(typeExpr)) base = "timestamp";
527     else if (/^date\b/.test(typeExpr)) base = "date";
528     else if (/^jsonb\b/.test(typeExpr)) {
529         const typeAnnotation = typeExpr.match(/\. \s*$type&lt;([^\&gt;]+)&gt; /);
530         const inner = typeAnnotation ? typeAnnotation[1].trim() : "";
531         if (inner === "string[]" || inner === "number[]") {
532             base = `jsonb \`${inner}\``;
533         } else {
534             base = "jsonb";
535         }
536     } else if (/^pgEnum\b/.test(typeExpr)) base = "enum";
537     // column defined via pgEnum variable: "someEnum("col")"
538     else if (/(\s*"["^"]+\s*\)/.test(typeExpr) && !typeExpr.startsWith("serial")) {

```

```

539     base = "enum";
540 }
541
542 const parts: string[] = [base];
543 if (isPK) parts.push("PK");
544 if (fkTarget) parts.push(`FK→${fkTarget}`);
545 if (isUnique && !isPK) parts.push("■");
546
547 return parts.join(" ");
548 }
549
550 /**
551  * Extract all pgTable(...) definitions from a TypeScript source string.
552  * Returns a list of { tableName, variableName, bodyText }.
553  */
554 function extractTableBlocks(src: string): Array<{ varName: string; tableName: string; body: string }>; {
555     const results: Array<{ varName: string; tableName: string; body: string }>; = [];
556
557     // Match: export const xyzTable = pgTable("actual_name", {
558     const declRe = /export\s+const\s+(\w+)\s*=\s*pgTable\s*\(\s*"([^"]+)"\s*,\s*\s*\{/g;
559
560     let m: RegExpExecArray | null;
561     while ((m = declRe.exec(src)) !== null) {
562         const varName = m[1];
563         const tableName = m[2];
564         const bodyStart = m.index + m[0].length;
565
566         // Find matching closing brace for the columns object
567         let depth = 1;
568         let i = bodyStart;
569         while (i <= src.length && depth > 0) {
570             if (src[i] === "{") depth++;
571             else if (src[i] === "}") depth--;
572             i++;
573         }
574         const body = src.slice(bodyStart, i - 1);
575         results.push({ varName, tableName, body });
576     }
577
578     return results;
579 }
580
581 /**
582  * Parse individual column definitions from a pgTable body string.
583  * Returns an array of { colName (DB name), rawTypeExpr }.
584  */
585 function parseColumns(body: string): Array<{ colName: string; rawTypeExpr: string }>; {
586     const cols: Array<{ colName: string; rawTypeExpr: string }>; = [];
587
588     // Split on lines that look like top-level column definitions.
589     // Pattern: leading whitespace, JS identifier, colon, then type call.
590     const lineRe = /\s{2,4}(\w+)\s*:\s*([\s\S]+?)(?=\s*$|\s*\\\/|$/gm;
591
592     let lm: RegExpExecArray | null;
593     while ((lm = lineRe.exec(body)) !== null) {
594         const jsKey = lm[1];
595         const rawExpr = lm[2].trim();
596
597         // Skip helper properties that aren't column definitions
598         if (jsKey === "primaryKey" || jsKey === "unique" || jsKey === "uniqueIndex") continue;
599         // Must start with a type function call
600         if (!/^((serial|integer|varchar|text|real|boolean|timestamp|date|jsonb|pgEnum|primaryKey|\w+Enum)\s*(\/.tes ...
601
602         // Extract the DB column name from the first string arg of the type call
603         const colNameMatch = rawExpr.match(/\(\s*"([^"]+)"/);

```



```

604     if (!colNameMatch) continue;
605
606     cols.push({ colName: colNameMatch[1], rawTypeExpr: rawExpr });
607 }
608
609 return cols;
610 }
611
612 // -----
613 // Main
614 // -----
615
616 function run() {
617     const files = readdirSync(SCHEMA_DIR)
618         .filter((f) => f.endsWith(".ts") && f !== "index.ts")
619         .map((f) => join(SCHEMA_DIR, f));
620
621     // First pass: collect varName → tableName mapping (needed for FK resolution)
622     const varToTable = new Map<string, string>();
623     const sources: Array<{ file: string; src: string }> = [];
624     for (const file of files) {
625         const src = readFileSync(file, "utf8");
626         sources.push({ file, src });
627         const blocks = extractTableBlocks(src);
628         for (const b of blocks) varToTable.set(b.varName, b.tableName);
629     }
630
631     // Second pass: parse all table definitions
632     const allTables: TableDef[] = [];
633     for (const { file, src } of sources) {
634         const blocks = extractTableBlocks(src);
635         for (const { varName, tableName, body } of blocks) {
636             const rawCols = parseColumns(body);
637             const columns: ColumnDef[] = rawCols.map(({ colName, rawTypeExpr }) => ({
638                 colName,
639                 typeSig: buildTypeSig(rawTypeExpr, varToTable),
640             }));
641             allTables.push({ tableName, columns, sourceFile: file });
642         }
643     }
644
645     // Build a lookup map
646     const tableMap = new Map<string, TableDef>();
647     for (const t of allTables) tableMap.set(t.tableName, t);
648
649     // Determine total table count
650     const totalTables = allTables.length;
651
652     // Helper: get Chinese description for a field
653     function fieldDesc(tableName: string, colName: string): string {
654         return (
655             FIELD_META[`${tableName}.${colName}`] ??
656             FIELD_META[`${colName}`] ??
657             colName
658         );
659     }
660
661     // Helper: render one table section. Returns empty string if the table no
662     // longer exists in the schema (e.g. after it has been deleted), so that the
663     // module list stays strictly in sync with the actual schema files.
664     function renderTable(tableName: string): string {
665         const def = tableMap.get(tableName);
666         if (!def) return "";
667
668         const meta = TABLE_META[tableName];

```

```

669     const displayName = meta?.zh ?? tableName;
670     const desc = meta?.desc ?? "";
671
672     const lines: string[] = [];
673     lines.push(`### \`${tableName}\` - ${displayName}`);
674     lines.push("");
675     if (desc) {
676         lines.push(desc);
677         lines.push("");
678     }
679     lines.push("| ■■ | ■■ | ■■ |");
680     lines.push("|---|---|---|");
681
682     for (const col of def.columns) {
683         const zh = fieldDesc(tableName, col.colName);
684         lines.push(`| \`${col.colName}\` | ${col.typeSig} | ${zh} |`);
685     }
686
687     const footnote = TABLE_FOOTNOTES[tableName];
688     if (footnote) {
689         lines.push("");
690         lines.push(footnote);
691     }
692
693     lines.push("");
694     lines.push("---");
695     lines.push("");
696     return lines.join("\n");
697 }
698
699 // Assemble document
700 const parts: string[] = [];
701 parts.push(`# ■■■ ■■■■■■■■`);
702 parts.push("");
703 parts.push(`&gt; ■ **${totalTables} ■■*■■■■■■■■`);
704 parts.push("");
705 parts.push("---");
706 parts.push("");
707
708 const renderedTables = new Set<string>();
709
710 for (const mod of MODULES) {
711     // Only emit tables that actually exist in the current schema
712     const existingTables = mod.tables.filter((tbl) => tableMap.has(tbl));
713     if (existingTables.length === 0) continue;
714
715     parts.push(`## ${mod.title}`);
716     parts.push("");
717     for (const tbl of existingTables) {
718         parts.push(renderTable(tbl));
719         renderedTables.add(tbl);
720     }
721 }
722
723 // Append any tables not listed in any module
724 const extras = allTables.filter((t) => !renderedTables.has(t.tableName));
725 if (extras.length > 0) {
726     parts.push("## ■■");
727     parts.push("");
728     for (const t of extras) {
729         parts.push(renderTable(t.tableName));
730     }
731 }
732
733 const output = parts.join("\n");

```

```

734   writeFileSync(OUTPUT_FILE, output, "utf8");
735   console.log(`✓ ███ ${OUTPUT_FILE}███ ${totalTables} ███`);
736 }
737
738 run();

```

scripts/src/hello.ts

```
1 console.log("Hello from @workspace/scripts");
```

scripts/src/seed.ts

```

1 import { db, pool } from "@workspace/db";
2 import {
3   usersTable, opcProfilesTable, demandsTable, bidsTable,
4   ordersTable, deliverablesTable, portfoliosTable, notificationsTable
5 } from "@workspace/db/schema";
6
7 async function seed() {
8   console.log("Seeding database...");
9
10  await db.delete(notificationsTable);
11  await db.delete(deliverablesTable);
12  await db.delete(ordersTable);
13  await db.delete(bidsTable);
14  await db.delete(demandsTable);
15  await db.delete(portfoliosTable);
16  await db.delete(opcProfilesTable);
17  await db.delete(usersTable);
18
19  const users = await db.insert(usersTable).values([
20    { nickname: "████████", phone: "13800138001", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=HCY", ...
21    { nickname: "████", phone: "13800138002", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=ZMY", rol ...
22    { nickname: "████", phone: "13800138003", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=LSQ", rol ...
23    { nickname: "████", phone: "13800138004", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=WYQ", rol ...
24    { nickname: "████", phone: "13800138005", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=CZH", rol ...
25    { nickname: "████", phone: "13800138006", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=LFH", rol ...
26    { nickname: "████", phone: "13800138007", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=ZWB", rol ...
27    { nickname: "████", phone: "13800138008", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=SHY", rol ...
28    { nickname: "██████", phone: "13800138000", avatar: "https://api.dicebear.com/7.x/initials/svg?seed=ADM", r ...
29  ]).returning();
30
31  const opcUsers = users.filter(u => u.role === "opc");
32  await db.insert(opcProfilesTable).values([
33    {
34      userId: opcUsers[0].id, level: "A",
35      bio: "██AI██████████Vibe Coding████AI██████5██████████",
36      skillTags: ["AI████", "Web██", "██████", "Vibe Coding"],
37      industryTags: ["AI██", "██", "██████"],
38      creditScore: 4.8, totalOrders: 28, completionRate: 96.4, avgRating: 4.9, totalEarnings: 385000, activity ...
39    },
40    {
41      userId: opcUsers[1].id, level: "B",
42      bio: "██PPT██████████████████████",
43      skillTags: ["PPT██", "████", "████", "████"],
44      industryTags: ["AI██", "████", "██"],
45      creditScore: 4.5, totalOrders: 19, completionRate: 94.7, avgRating: 4.7, totalEarnings: 156000, activity ...
46    },
47    {
48      userId: opcUsers[2].id, level: "A",
49      bio: "██████████████████████30+██████████",
50      skillTags: ["██████", "████", "████", "████"],
51      industryTags: ["████", "██", "██████"],
52      creditScore: 4.7, totalOrders: 35, completionRate: 97.1, avgRating: 4.8, totalEarnings: 290000, activity ...
53    },
54    {

```

```

55   userId: opcUsers[3].id, level: "B",
56   bio: "██████AI██████Python/JS██████",
57   skillTags: ["AI██████", "██████", "Web██████", "██████"],
58   industryTags: ["AI██████", "██████"],
59   creditScore: 4.3, totalOrders: 12, completionRate: 91.7, avgRating: 4.4, totalEarnings: 98000, activityS ...
60 },
61 {
62   userId: opcUsers[4].id, level: "C",
63   bio: "████OPC████AI██████████████████",
64   skillTags: ["PPT██████", "██████", "██████"],
65   industryTags: ["AI██████"],
66   creditScore: 4.0, totalOrders: 3, completionRate: 100, avgRating: 4.2, totalEarnings: 8500, activityScor ...
67 },
68 {
69   userId: opcUsers[5].id, level: "B",
70   bio: "██████████████████████████████████",
71   skillTags: ["AI██████", "Web██████", "██████", "██████"],
72   industryTags: ["██████", "██████", "██████"],
73   creditScore: 4.6, totalOrders: 22, completionRate: 95.5, avgRating: 4.6, totalEarnings: 210000, activity ...
74 },
75 ]);
76
77 const publisher = users.find(u => u.role === "publisher")!;
78
79 const demands = await db.insert(demandsTable).values([
80   {
81     demandNo: "JDB-202603-0001", title: "AI██████████",
82     type: "ai_education", description: "██████████K12████AI████████████████████████████████████████6██████████AI████████████████████",
83     skillTags: ["██████", "██████", "AI██████", "PPT██████"],
84     opcLevel: "A", budget: 65000,
85     deadline: "2026-05-15",
86     milestones: [
87       { name: "██████████", deadline: "2026-04-10", deliverableDesc: "████6██████████" },
88       { name: "██████", deadline: "2026-04-30", deliverableDesc: "██████████PPT" },
89       { name: "██████████", deadline: "2026-05-15", deliverableDesc: "██████████████████" },
90     ],
91     mode: "open", status: "published", isUrgent: false,
92     bidDeadline: new Date("2026-04-05T18:00:00Z"),
93     publisherId: publisher.id,
94   },
95   {
96     demandNo: "JDB-202603-0002", title: "██AI██████████",
97     type: "gov_training", description: "██████████AI████████████████████████████████████████2██████████+██████████",
98     skillTags: ["██████", "PPT██████", "AI██████", "██████"],
99     opcLevel: "B", budget: 20000,
100    deadline: "2026-04-20",
101    milestones: [
102      { name: "██████████", deadline: "2026-04-08", deliverableDesc: "██████████████████" },
103      { name: "██████", deadline: "2026-04-15", deliverableDesc: "██████████████████" },
104    ],
105    mode: "open", status: "published", isUrgent: true,
106    bidDeadline: new Date("2026-04-01T18:00:00Z"),
107    publisherId: publisher.id,
108  },
109  {
110    demandNo: "JDB-202603-0003", title: "AI██████████",
111    type: "ai_research", description: "████AI██████████3████████████████████████████████████████+██+██████████",
112    skillTags: ["██████", "██████████", "██████"],
113    opcLevel: "B", budget: 12000,
114    deadline: "2026-04-25",
115    milestones: [
116      { name: "██████", deadline: "2026-04-10", deliverableDesc: "██████████████████" },
117      { name: "██████", deadline: "2026-04-25", deliverableDesc: "██████████████████" },
118    ],
119    mode: "open", status: "published", isUrgent: false,

```

[illegible]

```
185         { name: "■■■■", deadline: "2026-04-15", deliverableDesc: "■■■■■■■■■■" },
186         { name: "■■■■", deadline: "2026-04-28", deliverableDesc: "■■■■PPT■■■■" },
187     ],
188     mode: "open", status: "published", isUrgent: false,
189     bidDeadline: new Date("2026-04-12T18:00:00Z"),
190     publisherId: publisher.id,
191 },
192 ]).returning();
193
194 const orders = await db.insert(ordersTable).values([
195     {
196         orderNo: "ORD-202603-0001",
197         demandId: demands[0].id, opcId: opcUsers[0].id, publisherId: publisher.id,
198         amount: 75000, opcShare: 45000, publisherShare: 22500, platformFee: 7500,
199         status: "in_progress",
200         milestones: demands[0].milestones,
201         deadline: "2026-05-15",
202     },
203     {
204         orderNo: "ORD-202602-0001",
205         demandId: demands[1].id, opcId: opcUsers[1].id, publisherId: publisher.id,
206         amount: 20000, opcShare: 12000, publisherShare: 6000, platformFee: 2000,
207         status: "pending_acceptance",
208         milestones: demands[1].milestones,
209         deadline: "2026-04-20",
210     },
211     {
212         orderNo: "ORD-202602-0002",
213         demandId: demands[2].id, opcId: opcUsers[2].id, publisherId: publisher.id,
214         amount: 12000, opcShare: 7200, publisherShare: 3600, platformFee: 1200,
215         status: "completed", rating: 4.8, reviewComment: "■■■■■■■■■■■■■■■■■■■■",
216         milestones: demands[2].milestones,
217         deadline: "2026-04-25",
218     },
219 ]).returning();
220
221 await db.insert(deliverablesTable).values([
222     {
223         orderId: orders[1].id, milestoneId: 1, title: "■■■■■■■■",
224         description: "■■2■■■■■■■■■■■■■■■■■■■■", status: "approved",
225     },
226     {
227         orderId: orders[1].id, milestoneId: 2, title: "■■■■PPT",
228         description: "■■■■8■■■■■■■■PPT", status: "submitted",
229     },
230 ]);
231
232 await db.insert(portfoliosTable).values([
233     {
234         userId: opcUsers[0].id, title: "■■■■AI■■■■",
235         type: "AI■■■■", description: "■■■■■■■■■■AI■■■■■■■■■■AI■■■■",
236         rating: 4.9, clientFeedback: "■■■■■■■■■■■■■■■■■■■■",
237     },
238     {
239         userId: opcUsers[0].id, title: "■■■■AI■■■■",
240         type: "AI■■", description: "■■■■■■■■■■AI■■■■■■■■■■■■■■■■■■■■",
241         rating: 4.8, clientFeedback: "■■■■■■■■■■■■■■■■■■■■",
242     },
243     {
244         userId: opcUsers[1].id, title: "AI■■■■■■■■",
245         type: "■■■■", description: "■■■■■■■■■■AI■■■■■■■■■■■■■■■■■■■■",
246         rating: 4.7, clientFeedback: "■■■■■■■■■■■■■■■■■■■■",
247     },
248     {
249         userId: opcUsers[2].id, title: "■■■■■■■■",
```

[illegible]

本源程序文档依据国家版权局软件著作权登记相关规范编制，
所有源代码均为接单吧 OPC撮合交易平台原创开发成果。

编制日期：2026-07-23